Each of these sections is effectively independent of the main content. Each section contains business logic (deciding which posts or ads to show), database access (loading the details of the posts), and rendering logic for how to display the data.

In chapter 7 you saw that you can use layouts and partial views to split the rendering of a view template into similar sections, but partial views aren't a good fit for this example. Partial views let you encapsulate view rendering logic but not business logic that's independent of the main page content. Instead, view components provide this functionality, encapsulating both the business logic and rendering logic for displaying a small section of the page. You can use DI to provide access to a database context, and you can test view components independently of the view they generate, much like MVC and API controllers. Think of them as being a bit like mini MVC controllers or mini Razor Pages, but you invoke them directly from a Razor view instead of in response to an HTTP request.

TIP View components are comparable to child actions from the legacy .NET Framework version of ASP.NET, in that they provide similar functionality. Child actions don't exist in ASP.NET Core.

View components vs. Razor Components and Blazor

In this book I focus on server-side rendered applications using Razor Pages and API applications using minimal APIs and web API controllers. .NET 7 also has a different approach to building ASP.NET Core applications: Blazor. I don't cover Blazor in this book, so I recommend reading *Blazor in Action*, by Chris Sainty (Manning, 2021).

Blazor has two programming models, client-side and server-side, but both approaches use *Blazor components* (confusingly, officially called Razor components). Blazor components have a lot of parallels with view components, but they live in a fundamentally different world. Blazor